Proposta de Resolução da Lista 1 de Sistemas Operacionais – 2026/1

Professor: Hélio Crestana Guardia

Monitor: Natanael Monteiro Pastore Antonioli

1. O que é um **Sistema Operacional**? Quais são suas **funções**?

O sistema operacional é um software de baixo nível responsável por mediar a integração entre o hardware e softwares de alto nível, bem como por fornecer, para o usuário, uma interface gráfica ou uma linha de comando com o qual este pode interagir.

Funções são:

- Dividir recursos de CPU e memória RAM entre programas;

- Controlar leitura e escrita de arquivos em discos;

- Receber e exibir dados de periféricos por meio de drivers;

- Gerenciar permissões conforme usuários.

2. Descreva o modo de operação de um computador de acordo com a arquitetura de *Von Neumann*.

A memória armazena dados e programas, a unidade lógica aritmética realiza operações, a unidade de controle busca por programas na memória e os executa através de endereços presentes nos registradores.

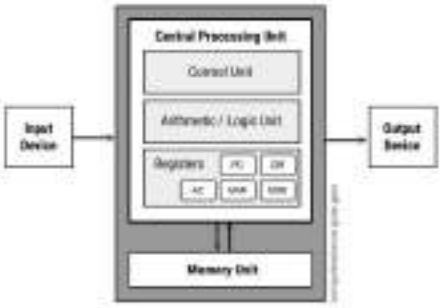
Em cada instrução, temos:

1. Busca, em que a unidade de controle usa o contador de programa (PC) para localizar a próxima instrução na memória.

2. Decodificação, em que a instrução é interpretada em termos de operação e dados necessários.

3. Execuções, em que a UlA executa a operação.

4. Armazenamento, em que o resultado é gravado na memória.



3. De que maneira são **representados** os processos em um sistema operacional?

Um processo é representado em um bloco de controle de processo (PCB), que possui:

- PID: Identificador único do processo;

- Estado: indica como o processo está, seja pronto, executando, finalizando ou bloqueado;

- Contador de programa: indica a próxima instrução a ser executada por meio do endereço;

- Conteúdos salvos de registradores;

- Ponteiros para dados na memória;

- Informações de escalonamento como prioridade, tempo de CPU e fila de execução.

4. O que é **contexto** de um processo?

É o conjunto completo de informações que define um processo em termos de CPU, memória e arquivos.

5. Como ocorrem as **trocas** de contexto?

Na troca de contexto, o sistema salva todo o contexto do processo atual e, então, carrega o contexto do processo seguinte. Assim, tudo o que a CPU "enxerga" é o panorama de cada processo e, após a troca de contexto, o processo seguinte é continuado do ponto em que parou na execução anterior.

6. Descreva estratégias usadas por sistemas operacionais para o **compartilhamento do tempo** de execução de um processador entre diversos processos.

O sistema operacional decide qual processo será executado, e em quanto tempo, por meio de algumas estratégias principais, sendo elas:

- FIFO (first in, first out): processos executados na ordem de chegada, sem preempção, é simples, mas causa esperas;

- SJF (shortest job first): executa, primeiro, o processo com menor tempo de execução, mas pode fazer com que processos longos nunca executem;

- Round robin: cada processo é executado em um tempo fixo, alternadamente, chamado quantum.

- Escalonamento por prioridade: cada processo recebe uma prioridade, e aqueles com maior prioridade executam primeiro. Para evitar que processos com pouca prioridade nunca executem, a prioridade aumenta com o tempo.

7. Descreva técnicas usadas pelo sistema operacional para **interagir** com os **controladores** de dispositivos.

Há algumas opções.

- Polling, em que consultas são feitas a cada período de tempo por disponibilidade, o que é ineficiente;

- Interrupções de E/S, em que se depende de um sinal enviado pelo controlador para disparar uma interrupção.

Se há múltiplos processos utilizando um dispositivo e deseja-se evitar que dados sejam corrompidos por escritas não esperadas, pode-se usar semáforos para gerir o acesso a um recurso compartilhado.

8. O que são **interrupções**? Descreva os tipos existentes e os eventos aos quais estão associados.

A interrupção suspende o funcionamento normal da CPU para que um evento urgente seja tratado.

Interrupções de E/S são enviadas por periféricos para notificar da leitura ou escrita em disco, de uma tecla pressionada ou de dados recebidos através da rede.

Interrupções de software são usadas para acessar serviços do SO, como em syscalls, que serão abordadas a seguir.

Interrupções involuntárias são causadas por erros de execução, como uma divisão por zero ou um acesso inválido à memória.

9. Como ocorre o **tratamento** de uma interrupção? Quais ações são desempenhadas pelo processador e quais são realizadas pelo Sistema Operacional?

É preciso:

1. Receber a sinalização da interrupção pelo controlador (hardware);
2. Concluir a instrução atual (software/hardware);
3. Salvar o estado atual e mudar de contexto (hardware);
4. Consultar a tabela de valores e desviar para a rotina de tratamento (hardware);
5. Executar a rotina de tratamento (software);
6. Restaurar o estado (hardware);

10. Como deve ser uma rotina de **tratamento de interrupção** no sistema operacional? Que ações devem ser executadas?

A rotina não pode ser blocante, deve ser curta e de tempo previsível, não deve envolver chamadas complexas e deve ser resistente a interrupções durante sua execução.

Essa rotina é responsável por ler os dados recebidos, escrevê-los em um buffer, enfileirar processos para tratamento desses dados fora da rotina e acordar processos que estejam esperando pelos dados lidos.

11. O que são **chamadas de sistema**? Como são implementados e de que maneira um programa de usuário tem acesso aos serviço do sistema operacional?

Chamadas de sistema permite que programas de usuário acessem funções do kernel de forma segura, sem acesso direto ao hardware. Para isso, uma biblioteca expõe aos programas de usuário um conjunto de funções comuns que, em seu interior, realizam syscalls.

Assim, o programador não precisa se preocupar com a segurança em baixo nível ou com os detalhes de suas chamadas particulares à arquitetura do sistema, já que a biblioteca se encarrega disso através das funções expostas.

12. Em um sistema operacional, o que é um programa *shell*? Como funciona?

Trata-se de uma interface responsável por receber comandos do usuário e convertê-los em ações para o sistema operacional.

13. Num *shell*, o que significa execução em *foreground* e execução em *background*? De que maneira o *shell* trata essas duas formas de execução de processos?

Um processo em foreground (primeiro plano) ocupa o termina, e o shell bloqueia até seu término. Quaisquer entradas de teclado vão para esse processo, e esse processo tem controle do terminal.

Um processo em background executa, mas não bloqueia o shell. O shell, por sua vez, volta imediatamente para o controle do usuário, que pode comandar outros processos.

14. Num sistema *unix*, como são identificados os processos?

Dois indicadores únicos são usados:

- PID: indicador único do processo;

- PPID: indicador único (PID) do processo pai, que o chamou;

15. Num *shell* de ambiente *unix*, para que serve o comando *ps*?

O comando lista processos ativos através do PID, bem como traz informações sobre estado, uso de CPU e processo pai.

16. Num sistema *unix*, o que são e para que servem os *sinais*? Dê exemplos de sinais.

Sinais são o equivalente das interrupções, mas para software. São usados para notificar um processo, que então tomará uma decisão como executar uma rotina de tratamento, finalizar ou ignorá-lo. Sinais são enviados pelo kernel, por outros processos ou pelo usuário no terminal.

17. Num sistema *unix*, para que serve o comando *kill*? Como são identificados os processos destinatários do sinais?

O comando kill envia sinais para processos, podendo solicitar o encerramento, ou causar o término forçado. Processos são identificados através do PID.

18. Considerando a definição de *chamadas de sistema* no padrão *POSIX*, explique de que forma as chamadas aos serviços do sistema operacional são implementadas. Como é o código inserido nos programas de usuário pelas chamadas de sistema e como os acessos aos serviços do SO são realizados em tempo de execução (*runtime*)?

A biblioteca usada por programas de usuário recebe um comando, valida os parâmetros apresentados, organiza os argumentos e, então, passam esses argumentos (já validados) para o kernel através do número da syscall. Nesse sentido, ela age como um encapsulador.

A CPU, então, executa essa instrução em modo privilegiado – que veio do usuário, mas não antes de passar pelo encapsulador e ser validada por ele.

19. Para que serve e como funciona a chamada ***fork***( )?
20. Diferencie as chamadas ***fork***( ) e ***vfork***( )

- pid_t **fork**(void);
- pid_t **vfork**(void);

A chamada fork cria um novo processo a partir de um processo já existente, sendo uma cópia lógica do processo pai (mesmos registradores, mesma rotina, mesmos arquivos).

A chamada vfork permite o compartilhamento (e não a cópia) do espaço de memória do pai. Ela é mais eficiente, mas é mais perigosa na medida em que permite que o filho interfira no espaço de memória no pai (ao invés de trabalhar somente com uma cópia dele).

19. Descreva a operação das chamadas **exec**( ). O que acontece com o contexto do processo que as executa? Quais informações do processo são preservadas após as chamadas?

- *int **execl**(const char *path, const char *arg, ...);*
- *int **execlp** (const char *file, const char *arg, ...);*
- *int **execle**(const char *path, const char *arg, ..., char * const envp[]);*
- *int **execv**(const char *path, char *const argv[]);*
- *int **execvp**(const char *file, char *const argv[]);*

Em uma chamada exec, um programa é substituído por outro, de maneira que o mesmo PID é preservado. Além disso, o PID do processo pai, o diretório e os arquivos abertos são preservados. Por outro lado, a pilha e dados globais são perdidos.

Temos:

- execl: o caminho completo é obrigatório.

- execpl: a busca ocorre no path, sem caminho completo.

- execle: permite definir um ambiente customizado para execução.

- execv: os argumentos são passados como um vetor, com caminho completo.

- execvp: os argumentos são passados como um vetor, sem caminho completo.

31. Descreva as funções ***wait***( ) e ***waitpid***( ). No que são diferentes?

- *pid_t **wait**(int *status);*
- *pid_t **waitpid**(pid_t pid, int *status, int options);*

Ambas são usadas para aguardar a conclusão de um processo filho, de forma que o processo pai possa, só após a conclusão, retomar sua execução.

Wait bloqueia o processo pai até qualquer filho terminar, e retorna o PID.

Waitpid bloqueia o processo pai até o filho especificado terminar, também retornando o PID.

32. O que são **sinais** para processos? Quais são os sinais definidos pelo padrão POSIX e a quais eventos estão associados? Que ações são definidas por padrão para o tratamento dos sinais?

Sinais funcionam como interrupções de software, enviando mensagens assíncronas para processos que sinalizam que um evento ocorreu. Os principais sinais são:

- SIGINT, uma interrupção do usuário.

- SIGTERM, uma solicitação de término.

- SIGKILL, para término forçado.

- SIGSTOP, para pausa no processo.

- SIGCONT, para retomada do processo.

- SIGSEGV, para tentativa de acesso em área proibida da memória.

- SIGABRT, para abortar execução.

- SIGCHLD, para mudança em processo filho.

- SIGALRM, para temporizador expirado.

Um sinal pode ser tratado pelo término do processo (o qual pode também seguir com um dump de memória), pela suspensão do processo ou pode ser simplesmente ignorado.

33. De que maneira o sistema operacional oferece o **tratamento** de sinais para os processos?

Pode-se realizar uma ação padrão já definida pelo sistema, ignorar o sinal ou executar um handler. Nesse último caso, o programa para, executa uma função já pré-definida e associada ao sinal, e então retoma.

34. Descreva as funções para manipulação de sinais:

- *sighandler_t **signal**(int signum, sighandler_t handler);*
- *int **sigaction**(int signum, const struct sigaction *act, struct sigaction *oldact);*
- *int **sigpending**(sigset_t *set);*
- *int **sigprocmask**(int how, const sigset_t *set, sigset_t *oldset);*

Utilizando signal, podemos especificar uma função a ser executada quando um sinal aparece.

Utilizando sigaction, podemos ter mais controle sobre essa especificação ao, por exemplo, bloquear outros sinais através de uma máscara.

Utilizando sigpending, obtemos o conjunto de sinais enviados, mas ainda não entregues.

Utilizando sigprocmask, especificamos a máscara de sinais, que é usada para bloquear sinais enquanto o handler executa.

35. Como é **enviado** um sinal para um processo?

Um sinal pode ser enviado através da chamada kill, usada entre processos; enviado para o próprio processo com raise; ou enviado pelo núcleo.

36. Em que momento uma função associada ao tratamento de um sinal por um processo é executada?

O handler só é executado quando o sinal é entregue (e, portanto, deixa de ser pendente), não quando o sinal é disparado.

37. Quais são as **políticas de escalonamento** definidas no *Linux?*
38. Descreva as políticas de escalonamento a seguir:

- SCHED_FIFO, SCHED_RR
- SCHED_OTHER e
- SCHED_BATCH

Temos:

- SCHED_Other, usada por padrão, que divide a CPU conforme tempo de CPU utilizado (chamado de prioridade nice).

- SCHED_Batch, usada para tarefas em lote.

- SCHED_Idle, de mais baixa prioridade possível, que só executa quando nenhuma outra tarefa está pronta.

- SCHED_FIFO, first in first out que, como o nome indica, executa as tarefas na ordem de chegada, podendo interrompê-las na chegada de uma tarefa de maior prioridade.

- SCHED_RR, que usa tempo real e prioridade fixa. É igual ao FIFO, mas usa um quantum de tempo para evitar inanição (starvation).

39. Descreva as funções a seguir, relacionadas com o controle do escalonamento de um processo:

- *int **getpriority**(int which, int who);*
- *int **setpriority**(int which, int who, int prio);*
- *int **sched_setscheduler**(pid_t pid, int policy, const struct sched_param *param);*
- *int **sched_getscheduler**(pid_t pid);*
- *int **sched_setparam**(pid_t pid, const struct sched_param *param);*
- *int **sched_getparam**(pid_t pid, struct sched_param *param);*
- *int **sched_get_priority_max**(int policy);*
- *int **sched_get_priority_min**(int policy);*
- *int **sched_rr_get_interval**(pid_t pid, struct timespec *tp);*

- getpriority é usada para obter a prioridade nice de um processo.

- setpriority permite sobrescrever a prioridade nice de um processo.

- setscheduler permitir definir a política de escalonamento a qual um processo está sujeito.

- getscheduler permite obter a política de escalonamento atual do process.

- setparam altera parâmetros de escalonamento, sem mudar a política.

- getparam retorna os parâmetros de escalonamento de um processo.

- get_priority_max retorna o maior valor de prioridade que uma política pode possuir.

- get_priority_min retorna o maior valor de prioridade que uma política pode possuir.

- rr_get_interval obtém o quantum na política de tempo real (SCHED_RR).

40. Para que serve a função *sched_yield( )*? Dê exemplo de uma situação de uso.

- *int **sched_yield**(void);*

Essa função faz com que o processo libere CPU e vá para o final da fila, mudando a ordem imediata de execução.

A função pode ser usada quando se detecta que um projeto não pode progredir, então, é prudente passa-lo ao final da fila na esperança que, até lá, outros processos permitam sua conclusão.

41. Em sistemas com múltiplos processadores (ou núcleos), para que serve o valor ***CPU affinity mask***, associado pelo sistema operacional a um processo (ou *thread*)?

- *int* **sched_setaffinity**(pid_t pid, unsigned int cpusetsize, cpu_set_t *mask);
- *int* **sched_getaffinity**(pid_t pid, unsigned int cpusetsize, cpu_set_t *mask);

  - *void CPU_CLR(int cpu, cpu_set_t *set);*
  - *int CPU_ISSET(int cpu, cpu_set_t *set);*
  - *void CPU_SET(int cpu, cpu_set_t *set);*
  - *void CPU_ZERO(cpu_set_t *set);*

Ao definir uma affinity mask, podemos delimitar em quais núcleos do processador um processo pode (1) ou não (0) executar. Assim, podemos evitar que haja migração de um núcleo para o outro (o que traz overhead), otimizando os processos.

42. Como são definidos os **limites** associados à execução de processos? Qual é a implicação de um limite ser atingido?

- *int **getrlimit**(int resource, struct rlimit *rlim);*
- *int **setrlimit**(int resource, const struct rlimit *rlim);*

```
struct rlimit {
  rlim_t rlim_cur;      // Soft limit
  rlim_t rlim_max;      // Hard limit (ceiling for rlim_cur)
};
```

A estrutura rlimit permite estabelecer uma quota de recursos que o sistema pode utilizar. Isso é feito através de um soft limit e um hard limit. O processo pode alterar seu soft limit livremente até o valor estabelecido pelo hard limit. O hard limit, por sua vez, só pode ser alterado pelo sistema.

43. Defina **recursos** que podem ser limitados para cada processo: RLIMIT_AS, RLIMIT_CORE, RLIMIT_CPU, RLIMIT_DATA, RLIMIT_FSIZE, RLIMIT_LOCKS, RLIMIT_MEMLOCK, RLIMIT_MSGQUEUE, RLIMIT_NICE, RLIMIT_NOFILE, RLIMIT_NPROC, RLIMIT_RSS, RLIMIT_RTPRIO, RLIMIT_SIGPENDING, RLIMIT_STACK, RLIMIT_OFILE

- AS: espaço máximo de endereçamento.

- CORE: máximo de arquivos de core dump.

- CPU: tempo máximo de CPU que o processo pode consumir.

- DATA: tamanho máximo do segmento de dados.

- FSIZE: tamanho máximo de arquivos.

- LOCKS: número máximo de locks de arquivos.

- MEMLOCK: máxima de memória que pode ser bloqueada.

- MSGQUEUE: máximo de bits em filas de mensagens pertencentes ao processo.

- NICE: maior limite que o processo pode atribuir para si mesmo na fila nice.

- NOFILE: número máximo de descritores de arquivo.

- NPROC: número máximo de processos.

- RSS: limite do conjunto residente (em termos de memória física).

- RTPRIO: prioridade máxima em políticas de tempo real.

- STACK: tamanho máximo da pilha de processos.

- OFILE: mesmo que NOFILE.

44. De que maneira são **contabilizados** pelo SO os recursos consumidos pelos processos?
45. Como é o **acesso** a essas informações e quais informações são providas pelo Sistema Operacional sobre a execução de um processo?

- *int **getrusage**(int who, struct rusage *usage);*

```
struct rusage {
  struct timeval ru_utime;  // user time used
  struct timeval ru_stime;  // system time used
  long ru_maxrss;   // maximum resident set size
  long ru_ixrss;    // integral shared memory size
  long ru_idrss;    // integral unshared data size
  long ru_isrss;    // integral unshared stack size
  long ru_minflt;   // page reclaims
  long ru_majflt;   // page faults
  long ru_nswap;    // swaps
  long ru_inblock;  // block input operations
  long ru_oublock;  // block output operations
  long ru_msgsnd;   // messages sent
  long ru_msgrcv;   // messages received
  long ru_nsignals;  // signals received
  long ru_nvcsw;    // voluntary context switches
  long ru_nivcsw;   // involuntary context switches
};
```

Ações específicas disparam um incremento do contador no kernel (como chamadas de leitura ou escrita, mensagens e sinais) bem como interrupções periódicas (ticks) verificam se o processo está ou não utilizando um recurso (como espaço de usuário ou espaço de núcleo).

Através da struct rusage, uma chamada obtém esse contador dentro do escopo especificado (o próprio processo ou os processos filhos), preenche a struct e a retorna.

46. De que maneira pode ser determinado o **tempo decorrido** (*elapsed time*) na execução de partes do código de uma aplicação? Exemplifique.

O relógio em tempo real subtrai o tempo de início e fim do processo.

O relógio em tempo de CPU, por sua vez, faz a mesma subtração, porém apenas quando a CPU está de fato sendo usada, ignorando, assim, o tempo gasto em espera.

47. Apresente um exemplo de código que ilustra uma forma de contabilizar o **tempo de CPU** gasto para a execução de um trecho de um programa.

```
#include <stdio.h>
#include <time.h>

int main() {

  // Ambos os relógios são iniciados

  struct timespec wall_start, wall_end;
  clock_t cpu_start, cpu_end;

  clock_gettime(CLOCK_MONOTONIC, &wall_start);
  cpu_start = clock();


  // === Soma dos números entre 1 e n ===

  int n;
  printf("Insira um numero : ");
  scanf("%d", &n);

  long long sum = 0;
  for (int i = 1; i <= n; i++)
    sum += i;

  // Ambos os relógios são encerrados
  cpu_end = clock();
  clock_gettime(CLOCK_MONOTONIC, &wall_end);

  double cpu_time  = (double)(cpu_end - cpu_start) / CLOCKS_PER_SEC;
  double wall_time = (wall_end.tv_sec  - wall_start.tv_sec)
             + (wall_end.tv_nsec - wall_start.tv_nsec) / 1e9;

  printf("Resultado : %lld\n", sum);
  printf("Tempo de CPU  : %.6f segundos\n", cpu_time);
  printf("Tempo real: %.6f segundos\n", wall_time);

  return 0;

}
```

48. O que é **IPC**? Descreva os principais mecanismos.

IPC é o conjunto de mecanismos que permite comunicação, incluindo troca de dados, entre processos.

- Entre o processo pai e os processos filhos, utiliza-se pipes anônimas, permitindo comunicação unidirecional.

- Entre processos não relacionados, utiliza-se pipes nomeadas, já que isso permite a identificação dos processos.

- Sockets permitem comunicação bidirecional, e são a base para sistemas distribuídos.

- Memória compartilhada define vários processos que usam a mesma região da memória, sendo mais rápido, mas exigindo sistemas de sincronização e bloqueio (como semáforos) para evitar incongruências.

- Sinais são notificações assíncronas que não transportam dados complexos.

49. O que são *threads*? Como são relacionadas com processos?

Um processo é uma instância de um programa em execução. Já uma thread existe dentro de um processo, e um processo pode ter várias threads. Uma thread compartilha memó ria e recursos do processo, mas tem seu próprio contador de programa, registradores e pilha.

A comunicação entre threads dentro de um processo ocorre justamente através da memória compartilhada.

50. Como são definidos os **contextos** das *threads*? Que informações são **comuns** entre *threads* e **particulares** para cada uma delas?

As informações particulares das threads são registradores, PC, pilha, estado da thread e ID da thread. Já o código, dados globais, heap, arquivos e recursos são compartilhados entre threads.

O contexto é o conjunto de informações mínimo necessário para retomar a execução da thread. Dados compartilhados não fazem parte do contexto.

51. Explique os conceitos a seguir, relacionados à implementação de *threads*: reentrância de código, *thread-safety*, *asynchronous-safety*.

- Reentrância de código diz respeito à possibilidade de executar um código (seja ele a própria thread, ou outras funções chamadas pela thread) simultaneamente, uma vez que não há um estado compartilhado.

- Thread safe é um atributo que define se o código funciona corretamente mesmo quando várias threads o executam ao mesmo tempo. Pode haver um estado compartilhado, mas mecanismos (como mutex, explicados adiante) são necessários.

- Async safe é um atributo que define se o código funciona corretamente se for interrompido por um sinal assíncrono. É mais restrito que reentrância, pois, aqui, os sinais para interrupção são arbitrários.

52. Descreva situações em que a programação usando *threads* é apropriada. Dê exemplos.

Quando há ganho na execução paralela de código. Um exemplo são simulações físicas com entidades independentes, como gotas d'água caindo e interagindo com um cenário, mas não entre si. Outro exemplo são renderizações de imagens, em que cada pixel age como uma entidade independente.

Experiência pessoal: ao desenvolver jogos na Unreal Engine, o conceito de paralelismo é extremamente importante. Efeitos de partículas (feitos a partir do Niagara) só funcionam graças ao paralelismo, e geradores de folhagem procedural (PCGs) podem ganhar tempo de geração se configurados para usar a GPU.

53. Descreva possíveis **modelos de programação** usando *threads*.

- Thread por tarefa: cada thread recebe uma tarefa própria para execução.

- Pool de threads: um conjunto fixo de threads é reutilizado para execução de tarefas, o que evita o overhead de destruição e criação de threads.

- Produtor/consumidor: algumas threads produzem dados (produtoras) e outras consumem (consumidoras). Isso envolve semáforos.

- Pipeline: o processamento é dividido em etapas sequenciais, e cada etapa é mapeada para uma thread.

- Mestre/trabalhador: uma thread mestre delega o trabalho para outras threads trabalhadoras, centralizando o controle.

54. Explique a função POSIX usada para a criação de *threads*:

- *int* ***pthread_create****(pthread_t * thread, const pthread_attr_t * attr, void * (*start_routine)(void *), void *arg);*

A função cria uma nova thread dentro de um processo. Os parâmetros são:

- O ponteiro onde a thread será armazenada.

- Atributos da thread.

- Função (trecho de código) que a thread executará ao iniciar.

- Argumentos passados para a função.

Um retorno 0 indica sucesso, um retorno diferente de 0 indica erro.

56. Explique a passagem de parâmetros e o tratamento de valores de retorno para funções associadas a *threads*.

A função executada pela thread é passada como um ponteiro (*start_routine). Os argumentos também são passados como um ponteiro (*arg). Ao passar múltiplos parâmetros, deve-se usar uma struct.

A função retorna void *, o que significa que a captura de valores de retorno precisa usar pthread_join.

57. Descreva os **atributos** ajustáveis na criação de *threads*.

- O estado de destacamento descreve se é possível sincronizar a thread. Se não, a thread libera ao terminar, sem o uso de Join.

- O tamanho da pilha é especificado. Uma pilha pequena pode causar overflow (já que não há mais espaço), ao passo que uma pilha grande pode causar desperdício de memória.

- Uma área de guarda (especificada por guard size) estabelece uma região protegida da pilha para detectar overflow.

- Podemos também definir a política de escalonamento.

58. Como são **encerradas** as execuções de *threads*?

A thread pode retornar normalmente (via return), pode retornar a partir dela mesma (chamando pthread_exit) ou pode ser cancelada por outra thread (com pthread_cancel).

Caso a thread filha deva ser aguardada (joinable), a thread que a chamou irá bloquear até a thread filha terminar. Caso não seja, o término e liberação de recursos ocorrem de forma imediata.

59. Quais são os mecanismos disponíveis para **sincronização** de *threads*?

Esses mecanismos permitem controlar o acesso de threads concorrentes a recursos compartilhados. No caso de uma região crítica da memória, isso é fundamental para evitar divergências que podem gerar um resultado incorreto.

- Mutex (mutual exclusion) permite que apenas uma thread acesse a região crítica por vez.

- Variáveis de condição permitem inativar threads até que a condição seja satisfeita. Assim, evita-se perda de recursos com threads bloqueadas aguardando liberação da região crítica.

- Semáforos permitem controle através da contagem. Um semáforo binário é um mutex (pois permite apenas 1 acesso simultâneo), mas semáforos podem permitir N acessos simultâneos.

- Barreiras estabelecem que threads irão esperar até que todas atinjam um objetivo.

- Locks de escrita e leitura permitem definir N leitores do espaço de memória, mas um único escritor. Assim, ao discriminar todo acesso à região crítica em acessos de leitura e escrita, pode-se evitar o bloqueio de threads que irão apenas ler a região crítica.

61. Como funciona o mecanismo ***mutex*** para sincronização de *threads*?
    - *pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;*
    - *int **pthread_mutex_init**(pthread_mutex_t *restrict mutex, const pthread_mutexattr_t *restrict attr);*
    - *int **pthread_mutex_destroy**(pthread_mutex_t *mutex);*
    - *int **pthread_mutex_lock**(pthread_mutex_t *mutex);*
    - *int **pthread_mutex_trylock**(pthread_mutex_t *mutex);*
    - *int **pthread_mutex_timedlock**(pthread_mutex_t *restrict mutex, const struct timespec *restrict abs_timeout);*
    - *int **pthread_mutex_unlock**(pthread_mutex_t *mutex);*

Em *pthread_mutex_t mutex* = *PTHREAD_MUTEX_INITIALIZER* definimos o mutex.

Então, pode-se definir o tipo de mutex:

- lock causa bloqueio da thread até que o mutex esteja liberado;

- trylock causa retorno imediato com erro.

- timedlock causa bloqueio até que o tempo seja esgotado, então causando retorno com erro.

Init irá chamar o mutex em tempo de execução. Uma liberação faz com que a thread adquira o mutex mas, se outra thread já possuir o mutex, então um dos procedimentos descritos acima irá ocorrer.

Unlock irá liberar o mutex, e, se houverem threads esperando pela sua liberação, uma delas será acordada.

Podemos chamar destroy para liberar os recursos do mutex desde que ele não esteja bloqueado.

62. Apresente um trecho de código que exemplifica o uso de *mutex*.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define ITERACOES 5

int contador = 0;
pthread_mutex_t mutex;

// A funcao executada por cada thread irá incrementar o contador ITERACOES vezes. Usaremos 2 threads, o que significa
que o resultado deve ser ITERACOES X 2.

void *incrementar(void *arg) {
  const char *nome = (const char *)arg;

  for (int i = 0; i < ITERACOES; i++) {
    // Para isso, ela trava o mutex.
    pthread_mutex_lock(&mutex);
    // Espera, simulando um trabalho longo
    int temp = contador;
    sleep(0);
    // Em seguida, incrementa em 1
    contador = temp + 1;
    printf("[%s] contador = %d\n", nome, contador);
    // E libera o mutex.
    pthread_mutex_unlock(&mutex);
  }
  return NULL;
}

// Essa é a função principal.
int main(void) {

  // Definimos as threads
  pthread_t t1, t2;

  // Inicializamos o mutex

  pthread_mutex_init(&mutex, NULL);

  // Criamos ambas as threads

  pthread_create(&t1, NULL, incrementar, "Thread-1");
  pthread_create(&t2, NULL, incrementar, "Thread-2");

  // Aguardamos ambas as threads terminarem,pois só assim o contador terá o valor necessário.

  pthread_join(t1, NULL);
  pthread_join(t2, NULL);

  // Verificamos o resultado.

  printf("\nValor final do contador: %d\n", contador);
  printf("Esperado:           %d\n", ITERACOES * 2);

  // Liberamos.

  pthread_mutex_destroy(&mutex);
  return 0;
}
```

Dica: experimente, agora, remover o mutex (mas deixe as threads) e verifique o valor. Ele é o esperado?

63. Para que servem e como funcionam as **barreiras** (***barriers***) associadas a *threads*?

- *int* ***pthread_barrier_init****(pthread_barrier_t *barrier, const pthread_barrierattr_t *attr, unsigned count);*
- *int* ***pthread_barrier_destroy****(pthread_barrier_t *barrier);*
- *int* ***pthread_barrier_wait****(pthread_barrier_t *barrier);*
- *int* ***pthread_barrierattr_init****(pthread_barrierattr_t *attr);*
- *int* ***pthread_barrierattr_destroy****(pthread_barrierattr_t *attr);*
- *int* ***pthread_barrierattr_getpshared****(const pthread_barrierattr_t *attr, int *pshared);*
- *int* ***pthread_barrierattr_setpshared****(pthread_barrierattr_t *attr, int pshared);*

Barreiras forçam que threads bloqueiem até que todas tenham atingido um certo ponto do código.

- wait define o ponto de sincronização, bloqueando cada thread até que todas estejam lá. Quando estão, todas são liberadas.

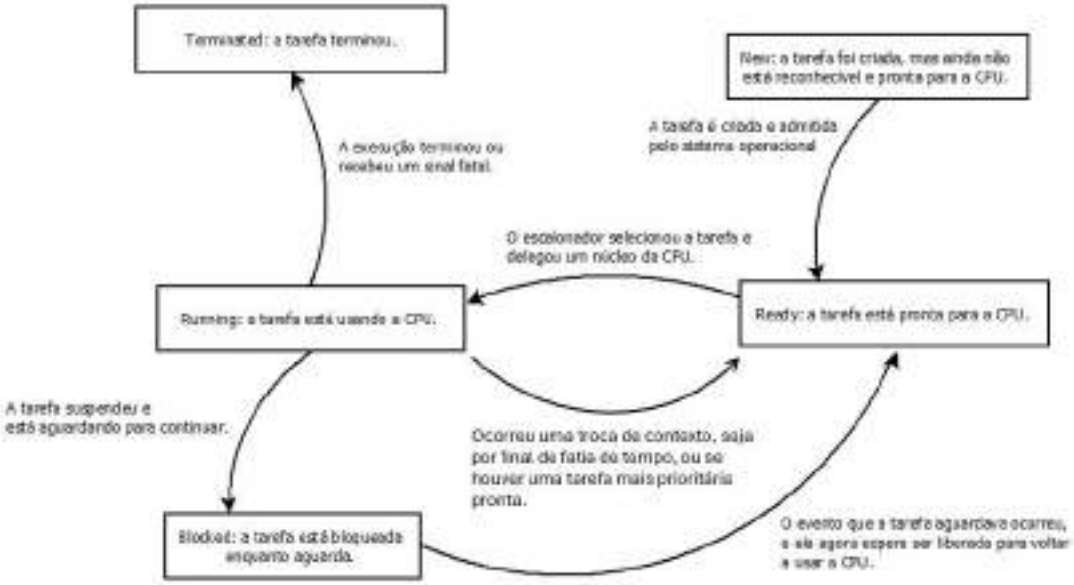
- init é responsável por definir a barreira ou seus atributos (no caso de barrierattr)

- destory é responsável por destruir a barreira ou seus atributos (no caso de barrierattr).

- setpshared e getpshared controlam e retornam (respectivamente) o compartilhamento da barreira, que pode ser apenas para threads de um mesmo processo, ou para threads de processos diferentes

64. Apresente um trecho de código com um exemplo de uso de **barreiras**.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 2
#define ITERACOES   3

pthread_barrier_t barreira;

// A função irá realizar 3 iterações nas quais trabalho é simulado pela função sleep. Para evidenciar a espera, cada thread demora mais que a anterior.

void *trabalhar(void *arg) {
   int id = *(int *)arg;

   for (int i = 0; i < ITERACOES; i++) {

      // Aqui, o trabalho é simulado

      printf("[Thread %d] fase 1, iteracao %d\n", id, i);
      sleep(id);

      // Aqui, a thread irá avisar antes de chegar na barreira. Só após essa mensagem é que outras threads poderão seguir.

      printf("[Thread %d] chegou na barreira (iteracao %d)\n", id, i);
      pthread_barrier_wait(&barreira);

     // Aqui, todas as threads atingiram a barreira, e veremos todas essas mensagens juntas. Isso ocorrerá, ao todo, 3 vezes.
      printf("[Thread %d] fase 2, iteracao %d\n", id, i);
   }

   return NULL;
}

int main(void) {

   pthread_t threads[NUM_THREADS];
   int ids[NUM_THREADS];

   // A barreira é incicializada.
   pthread_barrier_init(&barreira, NULL, NUM_THREADS);

   // As threads são criadas.
   for (int i = 0; i < NUM_THREADS; i++) {
      ids[i] = i + 1;
      pthread_create(&threads[i], NULL, trabalhar, &ids[i]);
   }

   // Aguardamos todas as threads terminarem
   for (int i = 0; i < NUM_THREADS; i++) {
      pthread_join(threads[i], NULL);
   }

   // Destruímos a barreira

   pthread_barrier_destroy(&barreira);
   return 0;
}
```

Dica: remova a barreira. A ordem das mensagens mudou?

65. Considerando o ciclo de vida de um programa, **esquematize os estados lógicos** em que uma tarefa pode estar e indique as possíveis **transições** entre esses estados, bem como **situações** (ou eventos) que levam a mudanças de estado.